
Understanding and Generalizing Confidence-Based Decoding in Masked Diffusion Models

Sijin Chen
Princeton University
chensj@princeton.edu

Chang Yu
Princeton University

Jiaxin Shi
Meta Superintelligence Labs

Jason M. Klusowski
Princeton University
jason.klusowski@princeton.edu

Abstract

Masked diffusion language models enable a more flexible generation order than their autoregressive counterparts, but raise the question of what unmasking order is optimal during sampling. Empirically, confidence-based unmasking heuristics perform strongly in practice, but the theoretical basis for their advantages remains unclear. In this work, we develop a theoretical account of confidence-based methods by studying a likelihood optimization problem that searches for the best unmasking policy to maximize sample likelihood. Under planning-depth reduction and distribution approximation, we show that confidence unmasking emerges as the (near-)optimal policy in our framework, justifying its empirical superiority. Building on the likelihood principle, we further propose a trainable policy that learns to optimize the global objective without the previous simplifications. Across experiments on multiple MDMs of different scales spanning text, code, and math generation benchmarks, the trained policy matches or exceeds the performance of confidence-based methods. Overall, our work demonstrates the potential of a likelihood optimization framework for analyzing and developing efficient sampling methods for diffusion language models.

1 Introduction

Diffusion language models [24, 42, 13, 34] recently emerged as a potential alternative to their autoregressive counterparts for language modeling with advances in discrete diffusion [1, 20, 29, 32, 26]. These models conduct training and inference by leveraging a forward-backward pair of continuous-time Markov chains (CTMC) evolving over continuous time $t \in (0, 1)$ in a discrete state space. In the forward (noising) process, a training data sample is gradually corrupted into pure noise; then the model aims to learn its reverse, i.e. the backward (denoising) process, that refines a random draw from pure noise step by step back to data.

The CTMC formulation takes a specific form in masked diffusion models (MDMs) [32, 29], currently one of the most representative discrete diffusion schemes for language modeling. In the forward process of MDMs, each token in the input sequence independently transitions to a special mask token M at the same rate, and remains in M thereafter. At inference, the backward process begins with an all-mask sequence and performs random unmasking, converting each M to a predicted text token. In this way, compared to autoregressive models that maintain a left-to-right ordering, MDMs offer enhanced flexibility in training and generation order.

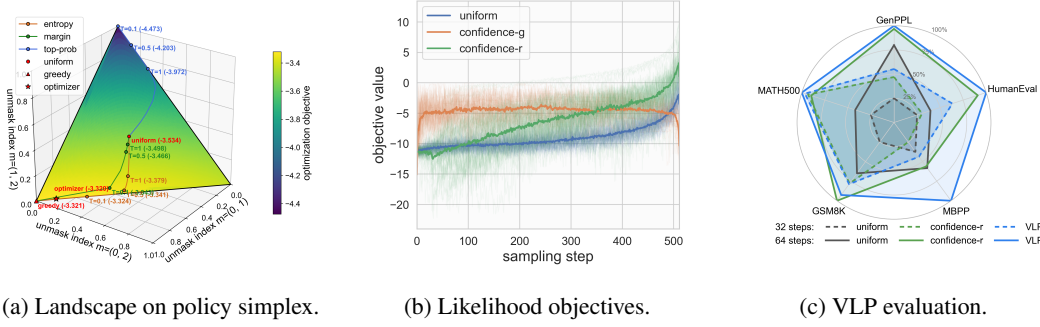


Figure 1: **(a)**: The landscape of the local likelihood maximization problem (7) over the policy simplex, and different unmasking methods with their attained objective values. **(b)**: Monte Carlo estimation of per-step local objective values (7) attained by different unmasking methods in text generation with RADD [27]. **(c)**: Selected evaluation results (see Table 1 and 2 for full version) of our likelihood-trained policy, VLP, compared to uniform and confidence unmasking methods in RADD text generation, and Dream-7B [42] code and math generation. Data are rescaled between the best and worst performance among the compared methods. [Jiaxin: can we increase the font size to be visible? May consider moving (c) to a separate figure.] [Sijin: Increased font size. Currently tend to keep (c) here because we have main results in the table and (c) is just for a quick introduction?]

Nonetheless, sampling from the backward process introduces several practical subtleties. For fast sampling, it is desirable to take large time steps Δt , unmasking multiple tokens per step. However, the ideal continuous-time process requires $\Delta t \rightarrow 0$ to guarantee simulation accuracy. Since unmasking can in principle be conducted in any order, this raises the question: which tokens should we choose to unmask at each step?

Standard MDM training does not answer this question because it only learns a continuous-time transition rate; nevertheless, several proposals exist. A first idea is to assign each token an equal probability of being unmasked at each sampling step, known as the τ -leaping strategy [6] or *uniform* unmasking. The motivation for this approach is that in the continuous-time limit, each token position has the same unmasking rate, mirroring the symmetry of the forward process. However, this method usually underperforms in empirical studies of MDM inference [44].

Another method that works better in practice is *confidence*-based unmasking [17, 25, 42]. Specifically, it prioritizes positions where the model’s predicted vocabulary distribution exhibits higher confidence. For example, in Figure 2, the left masked token has prediction $p(A) = 0.9$, $p(B) = 0.1$, indicating higher model confidence than the right position with $p(A) = p(B) = 0.5$; the left token is therefore unmasked first. Here, confidence can be quantified by various metrics such as negative entropy, top probability, and the margin between the top-2 probabilities; see details in Section 2.

1.1 Theoretical Understanding of Confidence

Despite their empirical success, the design of confidence-based methods remains largely heuristic, and the reasons for this success are not fully understood. Although earlier literature provides some intuition that these methods plan for a good reasoning path [17] and a concurrent work establishes its convergence property [5], their advantages over the uniform scheme remains theoretically unclear. This is the first question we aim to address in this paper: *Is confidence unmasking provably better for the time-discretized backward process?*

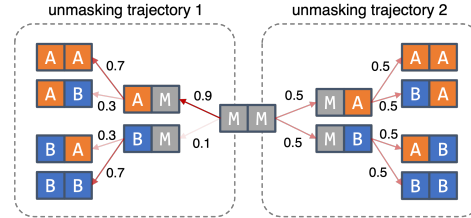


Figure 2: Two different sampling trajectories of decoding a length-2 sequence on a simple vocabulary $\{A, B\}$. Confidence-based unmasking indicates that trajectory 1 is preferred. [Jiaxin: Let’s tweak the example such that the two tokens are not independent? Otherwise the order is not relevant] [Sijin: Changed distribution.]

In the first half of this paper, we formulate an optimization problem that searches for the best unmasking policy π — the random mapping from the current sampling state to the index set unmasked at the next step. The objective of the problem is to maximize the sampling log-likelihood $\log p_0^{\theta, \pi}$ with respect to the discrete-time policy π and a parameterized backward process p_t^θ learned separately, i.e.

$$\max_{\pi} \mathbb{E}_{\mathbf{x}_0 \sim q_0} \log p_0^{\theta, \pi}(\mathbf{x}_0), \quad (1)$$

where q_0 is the unknown source data distribution. We show that, under a data-agnostic approximation and a local restriction of objective for tractability, entropy-based confidence unmasking emerges as a (near-)optimal policy (Section 4). This indicates a provable advantage of confidence unmasking over other methods in terms of sample likelihood, consistent with empirical observations. We also conduct numerical evaluations in synthetic and real text generation tasks to support our theoretical claims on the near-optimality of confidence-based unmasking methods (Figure 1a, 1b).

1.2 Unmasking Policy via Likelihood Optimization

At the same time, the reductions in our theory suggest that confidence unmasking is not optimal, as it does not maximize the original objective (1). This motivates us to revisit (1) and ask a second question: *Can we further develop advanced unmasking policies based on the original likelihood optimization problem?*

The second half of this paper is devoted to this extension from confidence to trainable policies. Specifically, we develop Variational Likelihood Policy (VLP), a prototypical training framework based on (1) with variational inference to optimize a neural-network-parameterized policy π , serving as an auxiliary trainable component to pretrained MDMs (Section 5). We train and evaluate VLP on differently scaled DLMs, including RADD [27], Dream-7B [42], and DiffuCoder-7B [13] across text, code, and math generation tasks (Figure 1c). The evaluation results indicate that our likelihood-trained policy, VLP, effectively learns unmasking policies and consistently matches or surpasses the performance of confidence-based methods across different models and tasks.

To summarize, in this paper, we propose a systematic framework based on likelihood optimization to analyze MDM unmasking methods. Our contribution lies in

- **Theoretical understanding of confidence:** We prove the theoretical advantages of confidence unmasking in terms of near-optimality in likelihood optimization (Section 4), supported by numerical analysis on the optimization objective in synthetic and real generation scenarios (Section 6.1; Figure 1a, 1b).
- **Principled design of trainable unmasking methods:** We design a likelihood-trained unmasking policy, VLP (Section 5), with variational inference and show its advantages through comprehensive evaluations across text, code, and math generation tasks on MDMs at different scales (Section 6.2; Figure 1c).

2 Background

2.1 Masked Diffusion Models

A continuous-time discrete diffusion model consists of a forward noising process and a backward denoising process, both continuous-time Markov chains. In the forward process, training samples are progressively corrupted into pure noise. The model learns the corresponding backward process that inverts this corruption, then generates new samples by running the backward process from noise.

Continuous-time Markov chain. We consider a discrete state space $\mathcal{X}$. A continuous-time Markov chain (CTMC) is a process $\mathbf{x}_t$ on $\mathcal{X}$ with $t \in [0, 1]$, starting from an initial data distribution $q_0(\mathbf{x}_0)$. A CTMC is characterized by a time-dependent transition rate matrix $Q_t \in \mathbb{R}^{|\mathcal{X}| \times |\mathcal{X}|}$. For distinct states $\mathbf{x}_t, \mathbf{y} \in \mathcal{X}$, $Q_t(\mathbf{x}_t, \mathbf{y}) \geq 0$ defines the instantaneous transition rate from $\mathbf{x}_t$ to $\mathbf{y}$, i.e., over an infinitesimal time interval $[t, t + \Delta t]$ the transition probability is given by

$$q_{t+\Delta t|t}(\mathbf{y}|\mathbf{x}_t) = \delta(\mathbf{x}_t, \mathbf{y}) + Q_t(\mathbf{x}_t, \mathbf{y})\Delta t + o(\Delta t), \quad (2)$$

where δ is the Kronecker delta. Equivalently, the marginal distribution $q_t \in \Delta(\mathcal{X})$ evolves according to the Kolmogorov ODE $dq_t/dt = q_t Q_t$. Note that the diagonal entries of Q_t should satisfy $Q_t(\mathbf{x}_t, \mathbf{x}_t) = -\sum_{\mathbf{x}_t \neq \mathbf{y}} Q_t(\mathbf{x}_t, \mathbf{y})$ to ensure that q_t sums to 1.

The masking and unmasking processes. In the forward process of MDMs with law q_t , the transition rate matrix is commonly parameterized as $Q_t = \sigma(t)Q$, where $\sigma(t)$ is a scalar noise schedule and Q is a constant rate matrix determined by the model design [6]. For MDMs, a specific Q is adopted so that over an infinitesimal interval $[t, t + \Delta t]$, the token-level transition probabilities are

$$q_{t+\Delta t|t}(y|x) = \begin{cases} \sigma(t) \Delta t + o(\Delta t), & y = \mathbf{M}, \\ 1 - \sigma(t) \Delta t - o(\Delta t), & y = x \in \mathcal{V}, \\ 0, & \text{otherwise,} \end{cases}$$

with a state space $\mathcal{V} \cup \{\mathbf{M}\}$. Letting $\bar{\sigma}(t) = \int_0^t \sigma(s) ds$ and $\alpha_t = e^{-\bar{\sigma}(t)}$, the distribution of a token at time t conditioned on the initial state x is given by

$$q_{t|0}(y|x) = \alpha_t \mathbf{1}[y = x] + (1 - \alpha_t) \mathbf{1}[y = \mathbf{M}].$$

Due to token-wise independence, the forward process for a length- L sequence is the product CTMC on $(\mathcal{V} \cup \{\mathbf{M}\})^L$, with the joint distribution

$$q_{t|0}(\mathbf{x}_t|\mathbf{x}_0) = \prod_{i=1}^L q_{t|0}(x_t^i|x_0^i).$$

The model aims to learn the time-reversal of the forward process q_t , denoted by p_t^θ : starting from an all-mask terminal state, we randomly unmask each position to a token in the vocabulary. We provide mathematical details in Appendix A.

2.2 Related Works on MDM Inference

Confidence-based unmasking. A common alternative to uniform unmasking in masked diffusion models (MDMs) is to prioritize positions where the model exhibits higher predictive confidence. Concretely, given the conditional token distribution $p_i(v) = p_t^\theta(x_{t-\Delta t}^i = v | \mathbf{x}_t)$ at position i , a confidence score c_i is computed and used to guide the unmasking order. Popular choices include:

- Negative entropy [42], $c_i = -\sum_v p_i(v) \log p_i(v)$;
- Top-1 probability [23], $c_i = \max_v p_i(v)$;
- Top-2 margin [17], $c_i = p_i(v_1) - p_i(v_2)$, where v_1, v_2 are the two most likely tokens at i .

Given these scores, unmasking strategies are typically either *greedy*, selecting the top- N positions with highest confidence, or *randomized*, for example by perturbing confidence scores before top- N selection [17] or sampling positions proportionally via a softmax transformation [42]. While simple and effective in practice, these approaches are largely heuristic and do not arise from an explicit optimization objective. Beyond fixed-size selection, Ben-Hamu et al. [3], Wu et al. [41] propose adaptive strategies on both which and how many tokens to unmask.

Trainable inference methods. Beyond fixed heuristics, several works treat the unmasking order as a trainable policy. We defer these discussions to Appendix A.

3 Unmasking Policy Optimization

3.1 Time-Discretized Process by Unmasking Policy

Let L be the sequence length. Denote $[L] = \{1, 2, \dots, L\}$, and $\Delta(\mathcal{S})$ the space of probability distributions on a set $\mathcal{S}$. Let K be a positive integer and $0 = t_0 < t_1 < \dots < t_K = 1$ form a partition of the interval $[0, 1]$. For efficient sampling, we aim to design a discrete-time unmasking process that operates on these timesteps $\{t_k\}$ in place of the learned MDM’s backward diffusion p_t^θ .

This can be done by introducing an unmasking policy. Formally, an unmasking policy is a mapping

$$\pi : (\mathcal{V} \cup \{\mathbf{M}\})^L \rightarrow \Delta(\text{Pow}([L]))$$

from a current sampling state to a probability distribution over the collection of all subsets of $[L]$. In practice, it is customary to further restrict the support of the output distribution to the collection of size- N subsets, where $N = L/K$. When L is not divisible by K , we set the size of the final step to the remainder of L divided by K .

Given a policy π and the continuous-time backward process p_t^θ of a trained MDM, we can build a discrete-time process $p_k^{\theta,\pi}$ as follows. At each step of $p_k^{\theta,\pi}$, given the current state $\mathbf{x}_k$, the policy π samples an index set $\mathcal{U}_k \subset [L]$ with probability $\pi(\mathcal{U}_k|\mathbf{x}_k)$, corresponding to a subset of the masked positions in $\mathbf{x}_k$. Then, the MDM’s prediction $p_{t_k}^\theta(\cdot|\mathbf{x}_k)$ at time t_k is used to fill in the newly unmasked tokens at $\mathcal{U}_k$, and the state transitions to $\mathbf{x}_{k-1}$. The transition probability from $\mathbf{x}_k$ to $\mathbf{x}_{k-1}$ is given by

$$p_k^{\theta,\pi}(\mathbf{x}_{k-1}|\mathbf{x}_k) = \pi(\mathcal{U}_k|\mathbf{x}_k)p_{t_k}^\theta(\mathbf{x}_{k-1}|\mathbf{x}_k) \quad (3)$$

where

$$p_{t_k}^\theta(\mathbf{x}_{k-1}|\mathbf{x}_k) = \prod_{i \in \mathcal{U}_k} p_k^\theta(x_{k-1}^i|\mathbf{x}_k), \quad \mathcal{U}_k = \{i : x_k^i = \mathbf{M} \neq x_{k-1}^i\}.$$

For the unmasking policy to be valid, we constrain the unmasking trajectories $\mathcal{U} = \{\mathcal{U}_1, \dots, \mathcal{U}_K\}$ produced by the policy to form a partition of the full index set:

$$\bigcup_{k=1}^K \mathcal{U}_k = [L]; \quad \mathcal{U}_k \cap \mathcal{U}_{k'} = \emptyset.$$

Both the uniform (τ -leaping) and confidence-based unmasking methods discussed above can be recovered by $p_k^{\theta,\pi}$ with appropriate choices of π .

3.2 The Maximum Likelihood Policy

Recall that a major question motivating this work is how we can theoretically compare different unmasking policies and search for the best one. This is made possible by connecting our discrete-time sampling process $p_k^{\theta,\pi}$ to maximum likelihood, a fundamental objective of generative modeling. Specifically, since $p_k^{\theta,\pi}$ is intended to recover the data distribution at the endpoint $k = 0$, its generative performance can be measured by the sample log-likelihood under the training data distribution q_0 . Under this criterion, the optimal policy is obtained by

$$\min_{\pi} -\mathbb{E}_{\mathbf{x}_0 \sim q_0} \log p_0^{\theta,\pi}(\mathbf{x}_0), \quad (4)$$

cf. (1). This objective is the original MDM training objective with the data marginal p_0^θ replaced by the policy-induced marginal $p_0^{\theta,\pi}$; this connection motivates our formulation.

The endpoint probability $p_0^{\theta,\pi}(\mathbf{x}_0)$ relates to the earlier sampling steps by marginalization:

$$p_0^{\theta,\pi}(\mathbf{x}_0) = \sum_{\mathbf{x}_1, \dots, \mathbf{x}_K} p_0^{\theta,\pi}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_K) = \sum_{\mathbf{x}_1, \dots, \mathbf{x}_K} \prod_{k=1}^K p_k^{\theta,\pi}(\mathbf{x}_{k-1}|\mathbf{x}_k).$$

Since the trajectories $(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_K)$ have nonzero probability only when each $\mathbf{x}_k$ is a masked outcome of $\mathbf{x}_{k-1}$, each such trajectory is fully determined by a sequence of unmasking sets $\mathcal{U} = (\mathcal{U}_1, \dots, \mathcal{U}_K)$ applied to $\mathbf{x}_K$. Thus equivalently,

$$p_0^{\theta,\pi}(\mathbf{x}_0) = \sum_{\mathcal{U}_1, \dots, \mathcal{U}_K} \prod_{k=1}^K p_k^{\theta,\pi}(\mathbf{x}_{k-1}|\mathbf{x}_k) = \sum_{\mathcal{U}_1, \dots, \mathcal{U}_K} \prod_{k=1}^K \pi(\mathcal{U}_k|\mathbf{x}_k) \prod_{i \in \mathcal{U}_k} p_{t_k}^\theta(x_{k-1}^i|\mathbf{x}_k),$$

due to (3). Note that for non-mask tokens it always holds that $x_{k-1}^i = x_0^i$. Plugging this into (4), we have the optimization problem

$$\min_{\pi} -\mathbb{E}_{q_0} \log \sum_{\mathcal{U}_1, \dots, \mathcal{U}_K} \prod_{k=1}^K \pi(\mathcal{U}_k|\mathbf{x}_k) \prod_{i \in \mathcal{U}_k} p_{t_k}^\theta(x_{k-1}^i|\mathbf{x}_k). \quad (5)$$

Directly solving (5) is intractable. The sum over $(\mathcal{U}_1, \dots, \mathcal{U}_K)$ has $\prod_{k=1}^K \binom{L_k}{N}$ terms, where L_k denotes the number of masked positions at step k . This count grows exponentially in K (e.g., $L!/(N!)^K$ when $L = KN$), and the log-sum-product structure prevents factorization across timesteps. This obstacle is structurally analogous to exact MAP decoding in autoregressive models, where global optimization is similarly intractable and local approximations (e.g. greedy decoding and beam search) are standard [9, 14, 36, 21]. In Sections 4 and 5 we develop two such approximations to (5): a single-step locality restriction that recovers confidence-based heuristics, and a variational lower bound that enables direct policy training.

4 Optimality of Confidence Heuristics

4.1 Locality Restriction

The objective (5) requires planning the entire unmasking trajectory $\mathcal{U}_1, \dots, \mathcal{U}_K$ globally, which is intractable. In this section, we instead minimize a local surrogate of (5) at each step k :

$$\min_{\pi(\cdot|\mathbf{x}_k)} -\mathbb{E}_{\mathbf{x}_0 \sim q_0(\cdot|\mathbf{x}_k)} \log \sum_{\mathcal{U}_k} \pi(\mathcal{U}_k|\mathbf{x}_k) \prod_{i \in \mathcal{U}_k} p_{t_k}^\theta(x_{k-1}^i|\mathbf{x}_k). \quad (6)$$

Here, the expectation is taken over the conditional distribution $q_0(\cdot|\mathbf{x}_k)$, i.e. the distribution over clean data $\mathbf{x}_0$ that could produce $\mathbf{x}_k$ via the forward masking process. The notation x_{k-1}^i denotes the value at position $i \in \mathcal{U}_k$ produced by the single-step unmasking from $\mathbf{x}_k$ to $\mathbf{x}_{k-1}$.

In this local problem, the summation is reduced to the index subsets $\mathcal{U}_k$, and instead of the full policy π we only need to optimize the single conditional distribution $\pi(\cdot|\mathbf{x}_k)$ by assigning probabilities $\pi(\mathcal{U}_k|\mathbf{x}_k)$ to each candidate subset $\mathcal{U}_k$. This is a convex optimization problem — by the concavity of log and the fact that expectation preserves convexity [4] — and is therefore much more tractable.

4.2 Deriving Confidence Unmasking

Another gap between heuristic unmasking strategies and our framework is that (6) requires precise knowledge of the conditional data distribution q_0 , while the heuristics are data-agnostic. When $q_0(\cdot|\mathbf{x}_k)$ is unknown, the natural (and only) approximation available is the MDM’s own prediction at time t_k :

$$\min_{\pi(\cdot|\mathbf{x}_k)} -\mathbb{E}_{\mathbf{x}_0 \sim p_{t_k}^\theta(\cdot|\mathbf{x}_k)} \log \sum_{\mathcal{U}_k} \pi(\mathcal{U}_k|\mathbf{x}_k) \prod_{i \in \mathcal{U}_k} p_{t_k}^\theta(x_{k-1}^i|\mathbf{x}_k). \quad (7)$$

4.2.1 Greedy Confidence Unmasking

Define entropy at index i to be

$$H_k(i) = - \sum_{v \in \mathcal{V}} p_{t_k}^\theta(x_{k-1}^i = v|\mathbf{x}_k) \log p_{t_k}^\theta(x_{k-1}^i = v|\mathbf{x}_k).$$

We have the following result on the optimality of greedy confidence unmasking based on negative entropy (cf. Section 2.2).

Theorem 4.1 (Informal). *Consider a policy π that selects an index set of size at least N . Then, the optimal deterministic policy π^* to (7) satisfies $\pi^*(\cdot|\mathbf{x}_k) = \mathbf{1}_{\mathcal{U}^*}$, where*

$$\mathcal{U}^* = \arg \max_{|\mathcal{U}|=N} \sum_{i \in \mathcal{U}} -H_k(i).$$

[Jiaxin: Do you mean there exists some $\mathcal{U}$ such that for any $\mathcal{U}'$? Also, I find this condition hard to interpret - we can improve the readability by simplifying the theorem as stating "the optimal deterministic policy is entropy minimized" and defer this extension to a remark that follows.] [Sijin: I prepared a simple statement in main text, and move the condition to appendix]

In words, π^* will deterministically output the index set $\mathcal{U}^*$ with the smallest total entropy (highest confidence) for the next step’s unmasking. Indeed, the optimality result generalizes to a certain subset of non-deterministic policies, and we provide a full statement and proof in Appendix B.2. We also leave remarks on the connection to prior entropy-based criteria and extensions to constrained confidence samplers in Appendix B.2.

4.2.2 Randomized Confidence Unmasking

In Theorem 4.1, the optimality of the greedy confidence policy relies on one of two specific conditions. In the more general case where neither condition holds, greedy confidence can be suboptimal and the optimal π^* becomes randomized. Nonetheless, in Theorem 4.2 we show that the optimal π^* can be characterized as a perturbation of the greedy point $\mathbf{1}_{\mathcal{U}^*}$. This resembles the randomized confidence unmasking used in practice (cf. Section 2.2), offering theoretical support for its empirical effectiveness.

Theorem 4.2. Let $\pi^*(\cdot|\mathbf{x}_k)$ be the optimal policy to (7), and $\mathcal{U}^*$ as defined in Theorem 4.1. Let $\mathbf{M}_k$ be the second moment matrix of the random variables $\prod_{i \in \mathcal{U}} p_{t_k}^\theta(x^i|\mathbf{x}_k)$, with $\mathcal{U}$ enumerating over the index subsets. Then,

$$\pi^*(\mathcal{U}^*|\mathbf{x}_k) \geq 1 - \lambda_{\max}(\mathbf{M}_k^{-1}) \|\mathbf{P}\mathbf{g}_k\|$$

where $\mathbf{g}_k$ is a vector indexed by $\mathcal{U}$ with

$$g_k^\mathcal{U} = \mathbb{E}_{\mathbf{x}_0 \sim p_{k-1}^\theta} \left[\frac{\prod_{i \in \mathcal{U}} p_{t_k}^\theta(x^i|\mathbf{x}_k)}{\prod_{i \in \mathcal{U}^*} p_{t_k}^\theta(x^i|\mathbf{x}_k)} \right],$$

and $\mathbf{P}$ is the projection to the hyperplane orthogonal to the all-one vector.

In words, while π^* is not greedy, it still assigns a high probability to the top-confidence subset $\mathcal{U}^*$ if the predicted distribution $p_{t_k}^\theta$ is well-conditioned. We defer the proof to Appendix B.4.

5 Building Trainable Unmasking Policy with Likelihood Principles

Our discussions above not only justify confidence unmasking as a reasonable approach for generating high-likelihood samples by minimizing (7), but also suggest room for developing other strategies by directly optimizing (1). This section presents VLP, a training and inference framework that learns a policy from (1) via variational inference. Relatedly, Ben-Hamu et al. [3] explicitly suggest learning a parameterized adaptive sampler optimizing a KL-based bound as a direction for future work; VLP can be seen as realizing this idea within our likelihood-maximization formulation (cf. (8)).

Recall that the log-likelihood in (1) can be expanded as

$$\log p_0^{\theta, \pi}(\mathbf{x}_0) = \log \sum_{\mathcal{U}} p_0^{\theta, \pi}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_K)$$

by marginalization, where (as in Section 3) the intermediate states $\mathbf{x}_1, \dots, \mathbf{x}_K$ are determined by the trajectory $\mathcal{U} = (\mathcal{U}_1, \dots, \mathcal{U}_K)$ and the data point $\mathbf{x}_0$ through the unmasking process. Training directly from this log-likelihood is intractable due to the exponential summation over $\mathcal{U}$.

Following Wang et al. [39], we introduce a variant of the evidence lower bound (ELBO) of the log-likelihood. Let $w(\cdot|\mathbf{x}_0)$ be an arbitrary discrete probability distribution over trajectories, with $w(\mathcal{U}|\mathbf{x}_0)$ denoting the probability of $\mathcal{U}$. By Jensen’s inequality,

$$\log p_0^{\theta, \pi}(\mathbf{x}_0) = \log \mathbb{E}_{\mathcal{U} \sim w(\cdot|\mathbf{x}_0)} \frac{p_0^{\theta, \pi}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_K)}{w(\mathcal{U}|\mathbf{x}_0)} \geq \mathbb{E}_{\mathcal{U} \sim w(\cdot|\mathbf{x}_0)} \log \frac{p_0^{\theta, \pi}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_K)}{w(\mathcal{U}|\mathbf{x}_0)}. \quad (8)$$

Maximizing this lower bound aligns the auxiliary trajectory distribution $w(\cdot|\mathbf{x}_0)$ with the model’s joint distribution $p_0^{\theta, \pi}(\mathbf{x}_0, \cdot)$, viewed as a measure over trajectories. Different choices of w yield different gaps between the bound and the exact log-likelihood.

To discourage the parameterized distributions from collapsing to degenerate solutions, we additionally introduce KL regularization — following standard practice in policy-based reinforcement learning [30] — between π_ϕ and a reference policy π_{ref} , and between w_ϕ and a reference w_{ref} , where π_{ref} implements confidence unmasking and w_{ref} is the uniform distribution over trajectories.

In VLP, both w and π are parameterized by neural networks w_ϕ and π_ϕ sharing weights ϕ . Substituting the parameterization into (8) gives the loss function used for training VLP:

$$\begin{aligned} \mathcal{L}(\phi) &:= -\mathbb{E}_{\mathbf{x}_0 \sim q_0} \mathbb{E}_{\mathcal{U} \sim w_\phi(\cdot|\mathbf{x}_0)} \log \frac{p_0^{\theta, \pi_\phi}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_K)}{w_\phi(\mathcal{U}|\mathbf{x}_0)} + \lambda \mathcal{R}(\phi), \\ \mathcal{R}(\phi) &:= \text{KL}(\pi_\phi \parallel \pi_{\text{ref}}) + \text{KL}(w_\phi \parallel w_{\text{ref}}). \end{aligned}$$

We defer the detailed description of the model architecture and training/inference algorithms used in VLP to Appendix C. [\[Jiaxin: Comment in appendix\]](#)

6 Experiments

In this section, we conduct two sets of experiments. In the first part, we provide empirical support for our optimality theory in Section 4 by evaluating, on synthetic and real text generation tasks, the local likelihood optimization objective (7) attained by different unmasking strategies. In the second part, we train unmasking policies with VLP and evaluate the resulting sampling performance on text, code, and math benchmarks across multiple MDMs. In addition to the main experiments, we also study the effect of KL regularization, temperature, and the per-step unmasking schedule.

6.1 Numerical Analysis on Likelihood Optimization

Landscape Visualization under Synthetic Distribution. We consider the task of unmasking 2 tokens from a length-3 sequence. There are exactly three possible unmasking actions: unmasking indices $\{0, 1\}$, $\{0, 2\}$, or $\{1, 2\}$, enabling a full visualization of the optimization landscape in 3-dimensional space. To simulate MDM predictions $p_t^\theta(\cdot | \mathbf{x}_t)$, we use a synthetic distribution described in Appendix D.1.1. In Figure 1a, we plot the landscape and each unmasking method on the policy simplex. The exact optimizer of (7) (★) is close to the entropy-based greedy point (▲), with closely matching objective values, illustrating Theorem 4.1 and Theorem 4.2. We provide additional results in Appendix D.1.1.

Objective Values in Real Text Generation. We also evaluate the local objective value (7) attained by different unmasking strategies on text generation with RADD [26], a pretrained MDM. We compare uniform unmasking, greedy confidence unmasking based on the top-2 probability margin, and its randomized counterpart. At each step, the objective (7) is approximated by Monte Carlo estimation. In Figure 1b, we set the generation length to 1024 and use 512 diffusion sampling steps. We observe that both confidence methods consistently achieve higher objective values than the uniform method, in alignment with our theory in Section 4. The objective value of greedy confidence method is initially the highest but would collapse at the final sampling steps, suggesting the importance of randomization in practice which can also be observed in Section 6.2. See Appendix D.1.2 for further results.

6.2 Evaluation of VLP

Text Generation. For text generation, we evaluate VLP on RADD [27], a pretrained MDM for unconditional text generation containing 162M parameters. We train the unmasking policy on OpenWebText [12] and evaluate generation quality using GPT-2 perplexity over sampling budgets from 16 to 256 steps. We compare against uniform unmasking and confidence unmasking based on top-2 probability margin, including both greedy and randomized variants. The detailed training and evaluation setup is provided in Appendix D.2.1. We also report sample entropy in Appendix D.2.2 to indicate that the performance gain does not compromise sample diversity.

Table 1: Average perplexity ($\downarrow$) of unconditional generations from RADD under different unmasking strategies and numbers of sampling steps. All generations have context length 1024.

steps	16	32	64	128	256
uniform	335.1	199.7	154.5	132.2	118.9
confidence-g	836.3	800.0	618.5	782.0	644.3
confidence-r	313.0	181.6	140.7	116.7	103.4
VLP	300.8	174.8	138.0	128.9	106.7

We report the GPT-2 perplexity of generated samples in Table 1. VLP substantially improves over uniform unmasking and matches or outperforms confidence-based methods, with the largest gains in the low-step regime. Randomized confidence unmasking is consistently stronger than greedy confidence unmasking, suggesting that deterministic high-confidence selection can be suboptimal.

Code and Math Generation. We evaluate VLP on code and math generation using two 7B-scale MDMs: Dream-7B [42] and DiffuCoder-7B [13]. We train the policy on the OpenCoder-SFT [15] and MetaMathQA [43] datasets, and evaluate on two code generation benchmarks — HumanEval [7] and MBPP [2], and two math generation benchmarks — GSM8K [10] and MATH500 [19]. The detailed setup is provided in Appendix D.2.

Results on both models are summarized in Tables 2 and 4 (deferred to Appendix D.2.2). Across Dream and DiffuCoder, VLP consistently matches or outperforms confidence-based unmasking, especially at small and moderate sampling budgets. Both VLP and the randomized confidence method show clear improvements over the uniform unmasking baseline. Greedy confidence unmasking often performs poorly, suggesting that deterministic confidence selection can be suboptimal. These results indicate that likelihood-trained unmasking policies can provide robust gains across various models, sampling budgets, and task domains.

Table 2: Results ($\uparrow$) of Dream-7B under different unmasking strategies and numbers of sampling steps across four benchmarks.

steps	HumanEval				MBPP				GSM8K				MATH500			
	16	32	64	128	16	32	64	128	16	32	64	128	16	32	64	128
uniform	8.5	17.6	19.5	23.2	18.6	25.6	27.6	30.8	23.4	30.0	34.9	35.4	10.4	12.8	13.8	16.8
confidence-g	3.0	5.5	14.0	20.7	1.4	1.6	6.4	13.6	12.8	29.4	42.1	40.7	3.6	7.0	15.0	12.0
confidence-r	15.2	18.3	26.2	27.4	25.2	24.4	27.4	32.0	32.7	36.5	39.2	39.6	15.2	16.8	16.6	18.2
VLP	17.0	22.6	27.4	29.3	22.8	26.2	31.6	32.8	33.6	36.6	38.3	38.2	15.6	17.0	17.2	19.4

6.3 Ablation Studies

We first ablate the **KL regularization strength** λ in RADD text generation experiments in Table 5. The effect of λ reveals that KL regularization consistently brings performance improvement to generation quality across multiple sampling budgets. We also study the effect of choosing different **diffusion steps** K in VLP training. In RADD text generation experiments in Table 6, training with $K = 32$ gives the strongest overall performance, achieving the best perplexity at most evaluation steps. The results also suggest that VLP trained on a fixed K is able to work on a broader range of different evaluation steps. Finally, Table 7 reports the performance of VLP under different choices of **sampling temperature** through evaluations. Appendix D.2.2 provides further details on these studies.

7 Conclusion

In this work, we study inference in masked diffusion language models through a likelihood-based formulation that treats the unmasking order as a policy to be optimized. Under local planning and distribution-approximation reductions, this framework explains confidence-based unmasking as an near-optimal policy, with numerical evidence supporting the likelihood-optimality view. Building on the same principle, we introduced Variational Likelihood Policy (VLP), a trainable unmasking policy optimized by a variational lower bound, and showed that it matches or improves over uniform and confidence-based unmasking across text, code, and math generation tasks and multiple MDMs at different scales.

This work still has several limitations. The theoretical derivation relies on local reductions and approximation assumptions, while the design of VLP is prototypical and can face inefficiency and instability in training. Future work may improve the optimization of VLP, explore joint training with the underlying diffusion model, and extend the likelihood principle to other discrete diffusion models beyond MDMs.

Acknowledgments

Jason M. Klusowski is grateful for support from the National Science Foundation through NSF CAREER DMS-2239448 and the Alfred P. Sloan Foundation through a Sloan Research Fellowship. Sijin Chen acknowledges support from Princeton University’s Gordon Wu Fellowship. The work reported on in this paper was substantially performed using Princeton University’s Research Computing resources.

References

- [1] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [3] Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *Advances in Neural Information Processing Systems*, 2025.
- [4] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, 2004. URL <https://web.stanford.edu/~boyd/cvxbook/>.
- [5] Changxiao Cai and Gen Li. Confidence-based decoding is provably efficient for diffusion language models. *arXiv preprint arXiv:2603.22248*, 2026.
- [6] Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems*, 35:28266–28279, 2022.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [8] Shirui Chen, Jiantao Jiao, Lillian J. Ratliff, and Banghua Zhu. dultra: Ultra-fast diffusion language models via reinforcement learning. *arXiv preprint arXiv:2512.21446*, 2025.
- [9] Sijin Chen, Omar Hagrass, and Jason M. Klusowski. Decoding game: On minimax optimality of heuristic text generation strategies, 2025. URL <https://arxiv.org/abs/2410.03968>.
- [10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [11] Thomas M Cover. Log optimal portfolios. In *Chapter in “Gambling Research: Gambling and Risk Taking,” Seventh International Conference*, volume 4, 1987.
- [12] Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. Openwebtext corpus. <http://Skylion007.github.io/OpenWebTextCorpus>, 2019.
- [13] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffucoder: Understanding and improving masked diffusion models for code generation, 2025. URL <https://arxiv.org/abs/2506.20639>.
- [14] Alex Graves. Sequence transduction with recurrent neural networks, 2012. URL <https://arxiv.org/abs/1211.3711>.
- [15] Siming Huang, Tianhao Cheng, Jason Klein Liu, Jiaran Hao, Liuyihan Song, Yang Xu, J. Yang, J. H. Liu, Chenchen Zhang, Linzheng Chai, Ruifeng Yuan, Zhaoxiang Zhang, Jie Fu, Qian Liu, Ge Zhang, Zili Wang, Yuan Qi, Yinghui Xu, and Wei Chu. Opencoder: The open cookbook for top-tier code large language models. 2024. URL <https://arxiv.org/pdf/2411.04905>.
- [16] Metod Jazbec, Theo X Olausson, Louis Béthune, Pierre Ablin, Michael Kirchhof, Joao Monterio, Victor Turrise, Jason Ramapuram, and Marco Cuturi. Learning unmasking policies for diffusion language models. *arXiv preprint arXiv:2512.09106*, 2025.

- [17] Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham M. Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=DjJmre5IkP>.
- [18] Seo Hyun Kim, Sunwoo Hong, Hojung Jung, Youngrok Park, and Se-Young Yun. Klass: Kl-guided fast inference in masked diffusion models. *arXiv preprint arXiv:2511.05664*, 2025.
- [19] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The twelfth international conference on learning representations*, 2023.
- [20] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 32819–32848. PMLR, 2024.
- [21] Clara Meister, Ryan Cotterell, and Tim Vieira. If beam search is the answer, what was the question? In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2173–2185, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.170. URL <https://aclanthology.org/2020.emnlp-main.170>.
- [22] Viacheslav Meshchaninov, Egor Shibaev, Artem Makoian, Ivan Klimov, Danil Sheshenya, Andrei Malinin, Nikita Balagansky, Daniil Gavrilov, Aibek Alanov, and Dmitry Vetrov. Guided star-shaped masked diffusion. *arXiv preprint arXiv:2510.08369*, 2025.
- [23] Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. Scaling up masked diffusion models on text, 2025. URL <https://arxiv.org/abs/2410.18514>.
- [24] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- [25] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL <https://arxiv.org/abs/2502.09992>.
- [26] Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. *arXiv preprint arXiv:2406.03736*, 2024.
- [27] Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data, 2025. URL <https://arxiv.org/abs/2406.03736>.
- [28] Fred Zhangzhi Peng, Zachary Bezemek, Sawan Patel, Jarrid Rector-Brooks, Sherwood Yao, Avishek Joey Bose, Alexander Tong, and Pranam Chatterjee. Path planning for masked diffusion model sampling. *arXiv preprint arXiv:2502.03540*, 2025.
- [29] Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.
- [30] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [31] Jiaxin Shi, Yuhao Zhou, Jessica Hwang, Michalis K. Titsias, and Lester Mackey. Gradient estimation with discrete Stein operators. *Advances in Neural Information Processing Systems*, 35:25829–25841, 2022.
- [32] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. *Advances in neural information processing systems*, 37:103131–103167, 2024.
- [33] Andy Shih, Dorsa Sadigh, and Stefano Ermon. Training and inference on any-order autoregressive models the right way, 2022. URL <https://arxiv.org/abs/2205.13554>.
- [34] Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuechang Yang, Hongli Yu, Xingwei Qu, Yuwei Fu, Jing Su, Ge Zhang, Wenhao Huang, Mingxuan Wang, Lin Yan, Xiaoying Jia, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Yonghui Wu, and Hao Zhou. Seed diffusion: A large-scale diffusion language model with high-speed inference, 2025. URL <https://arxiv.org/abs/2508.02193>.

- [35] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.
- [36] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/file/a14ac55a4f27472c5d894ec1c3c743d2-Paper.pdf.
- [37] Gilad Turok, Chris De Sa, and Volodymyr Kuleshov. Duel: Exact likelihood for masked diffusion via deterministic unmasking. *arXiv preprint arXiv:2603.01367*, 2026.
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [39] Zhe Wang, Jiaxin Shi, Nicolas Heess, Arthur Gretton, and Michalis Titsias. Learning-order autoregressive models with application to molecular graph generation. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=EY6pXIDi3G>.
- [40] Stephen J Wright and Benjamin Recht. *Optimization for data analysis*. Cambridge University Press, 2022.
- [41] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding, 2025. URL <https://arxiv.org/abs/2505.22618>.
- [42] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025. URL <https://arxiv.org/abs/2508.15487>.
- [43] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2023.
- [44] Yixiu Zhao, Jiaxin Shi, Feng Chen, Shaul Druckmann, Lester Mackey, and Scott Linderman. Informed correctors for discrete diffusion models. *arXiv preprint arXiv:2407.21243*, 2024.

A Additional Background and Preliminaries

Backward unmasking process of MDMs. It is known that the time reversal of a CTMC with rate Q_t is also a CTMC, whose infinitesimal transition probability takes a form analogous to (2):

$$q_{t-\Delta t|t}(\mathbf{y}|\mathbf{x}_t) = \delta(\mathbf{x}_t, \mathbf{y}) + \tilde{Q}_t(\mathbf{x}_t, \mathbf{y})\Delta t + o(\Delta t). \quad (9)$$

Here, $\tilde{Q}_t$ is an unknown transition rate. Specifically for MDMs, the reverse transition is an unmasking process [20, 6, 27], where we begin from an all-mask sequence and randomly unmask each to a token in the vocabulary with the following token level transitions:

$$q_{s|t}(x_s^i|\mathbf{x}_t) = \begin{cases} 1, & x_t^i \neq \mathbf{M}, x_s^i = x_t^i, \\ \frac{1 - e^{-\bar{\sigma}(s)}}{1 - e^{-\bar{\sigma}(t)}}, & x_t^i = \mathbf{M}, x_s^i = \mathbf{M}, \\ \frac{e^{-\bar{\sigma}(s)} - e^{-\bar{\sigma}(t)}}{1 - e^{-\bar{\sigma}(t)}} q_{0|t}(x_s^i|\mathbf{x}_t), & x_t^i = \mathbf{M}, x_s^i \neq \mathbf{M}, \\ 0, & \text{otherwise.} \end{cases}$$

Therefore, the learning of the backward process reduces to the learning of the reverse rate $\tilde{Q}_t$ parameterized by a neural network with weight θ . The training can be implemented by various approaches, such as by minimizing score-based objectives [20, 27]. In this way, we have a parameterized approximation of the true backward rate $P_t^\theta \approx \tilde{Q}_t$, which leads to a marginal distribution p_t^θ at time t by solving the Kolmogorov ODE $dp_t^\theta/dt = p_t^\theta P_t^\theta$.

Trainable inference methods for MDMs. Beyond fixed heuristics, several works treat the unmasking order as a trainable policy. Some introduce a separate policy network to learn update sequences [28, 16, 8] or introduce an additional MDM training stage on the preferred trajectories [34], while others modify the backward process itself to encourage more structured updates, such as jointly updating semantically related variables [22] or incorporating corrective refinement steps for already unmasked tokens [44]. These methods demonstrate that learning the inference procedure can significantly improve sample quality, but most existing approaches rely on rewards (reinforcement learning) or ad hoc update rules. Based on sample likelihood maximization, Wang et al. [39] proposes a framework to learn the optimal decoding order for any-order autoregressive models [33], which are limited to single-token generation processes; our VLP extends this framework to multi-token unmasking in masked diffusion models. From a related likelihood-based perspective, Turok et al. [37] restrict attention to deterministic position selection and obtain exact test-time likelihoods within this class.

B Proofs

B.1 Preparation

Consider an indexing of subsets $j \mapsto \mathcal{U}_j$ for all the size- N subsets $\mathcal{U}_j$ of $[L]$. We denote $\boldsymbol{\pi} \in \mathbb{R}^{\binom{L}{N}}$ a probability vector such that $\pi_j := \pi(\mathcal{U}_j|\mathbf{x}_k)$. Similarly, we denote $\mathbf{z} \in \mathbb{R}^{\binom{L}{N}}$ a random vector such that $z_j := \prod_{i \in \mathcal{U}_j} p_{t_k}^{\theta, \pi}(x^i|\mathbf{x}_k)$, with the randomness $x^i \sim p_{t_k}^{\theta, \pi}(\cdot|\mathbf{x}_k)$. Then, (7) becomes the following optimization problem over the probability simplex Δ :

$$\min_{\boldsymbol{\pi} \in \Delta} f(\boldsymbol{\pi}) := -\mathbb{E} \log(\mathbf{z}^\top \boldsymbol{\pi}). \quad (10)$$

B.2 Proof of Theorem 4.1

We first state the complete version of Theorem 4.2 on a general non-deterministic policy.

Theorem B.1. *Consider a policy π that selects an index set of size at least N . If (1) π is a deterministic policy, or (2) there exists $\mathcal{U}$ such that for all $\mathcal{U}'$, $p_{t_k}^\theta$ satisfies*

$$\mathbb{E}_{x^i \sim p_{t_k}^\theta(\cdot|\mathbf{x}_k)} \left[\frac{\prod_{i \in \mathcal{U}'} p_{t_k}^\theta(x^i|\mathbf{x}_k)}{\prod_{i \in \mathcal{U}} p_{t_k}^\theta(x^i|\mathbf{x}_k)} \right] \leq 1$$

, then the optimal strategy π^* to (7) satisfies $\pi^*(\cdot|\mathbf{x}_k) = \mathbf{1}_{\mathcal{U}^*}$, where

$$\mathcal{U}^* = \arg \max_{|\mathcal{U}|=N} \sum_{i \in \mathcal{U}} -H_k(i).$$

Proof. **Condition (1).** If π is restricted to be a deterministic policy, then the optimal $\pi^* = e_j$ for some standard basis vector e_j . Taking this to (10),

$$\begin{aligned} f(\pi^*) &= -\mathbb{E} \log z_j = -\mathbb{E}_{\mathbf{x} \sim p_{t_k}^\theta(\cdot|\mathbf{x}_k)} \log \prod_{i \in \mathcal{U}_j} p_{t_k}^\theta(x^i|\mathbf{x}_k) \\ &= -\sum_{i \in \mathcal{U}_j} \sum_{v \in \mathcal{V}} p_{t_k}^\theta(x^i = v|\mathbf{x}_k) \log p_{t_k}^\theta(x^i = v|\mathbf{x}_k) = \sum_{i \in \mathcal{U}_j} H_k(i). \end{aligned}$$

Therefore, the minimum must be attained at the policy that deterministically outputs $\mathcal{U}^*$ with the smallest total entropy.

Condition (2). By Cover [11], the KKT optimality conditions state that for an optimal π^* ,

$$\mathbb{E} \left[\frac{z_j}{\mathbf{z}^\top \pi^*} \right] \begin{cases} = 1, & \text{if } \pi_j^* > 0, \\ \leq 1, & \text{if } \pi_j^* = 0. \end{cases}$$

Thus, if e_k is optimal for some k , then $\mathbb{E}[z_j/z_k] \leq 1$ for all j . Moreover, since there exists strictly feasible π in the probability simplex, Slater's condition holds and KKT optimality is necessary and sufficient. Thus an optimal π^* must be deterministic, and we have the same result as in Condition (1). $\square$

Remark B.2. A related entropy criterion appears independently in Ben-Hamu et al. [3]: their EB-Sampler unmask token subsets satisfying an entropy-difference bound $\sum_{l \in \mathcal{U}} H_k(l) - \max_{l \in \mathcal{U}} H_k(l) \leq \gamma$, motivated as an upper bound on joint dependence error in a KL-based analysis of MDM sampling. Theorem 4.1 provides a complementary justification: under our locality restriction, entropy-greedy unmasking is the optimal stochastic policy for likelihood maximization, giving a likelihood-theoretic foundation for the use of entropy as the unmasking signal.

Remark B.3. The optimization framework underlying Theorem 4.1 naturally extends to other confidence-based heuristic samplers by augmenting (7) with additional data-dependent constraints on the policy. Provided such constraints act separably on the feasible simplex of $\pi(\cdot|\mathbf{x}_k)$ and preserve convexity, the KKT analysis carries through on the restricted region, and the optimal policy reduces to entropy-greedy unmasking within the constrained subset. We illustrate this with KLASS [18] in Appendix B.3.

B.3 Worked Example: KLASS as a Constrained Instance

We illustrate how the framework underlying Theorem 4.1 extends to other confidence-based heuristic samplers, using KLASS [18] as a representative example. KLASS augments confidence unmasking with a KL-stability filter: at each step k , it scores each masked position by the KL divergence between the predicted vocabulary distribution at $\mathbf{x}_k$ and predictions from a recent history window, and restricts unmasking to the stable subset

$$S_k = \{i \in [L] : x_k^i = \mathbf{M}, \text{ position } i \text{ is KL-stable over the history window}\} \subseteq [L].$$

Within this stable set, KLASS unmask the highest-confidence positions.

In our framework, this restriction amounts to constraining the support of $\pi(\cdot|\mathbf{x}_k)$ to subsets $\mathcal{U} \subseteq S_k$. Following the indexing of Appendix B.2, let $J(S_k) := \{j : \mathcal{U}_j \subseteq S_k\}$, and let

$$\Delta_{S_k} := \{\pi \in \Delta : \pi_j = 0 \text{ for all } j \notin J(S_k)\}$$

denote the induced sub-simplex. The KLASS-constrained version of (7) is then

$$\min_{\pi \in \Delta_{S_k}} f(\pi) = -\mathbb{E} \log(\mathbf{z}^\top \pi), \quad (11)$$

which differs from (10) only in its feasible set.

Proposition B.4. *Under the conditions of Theorem 4.1, with $|S_k| \geq N$, the optimal solution to (11) is $\pi^* = e_{j^*}$ where*

$$\mathcal{U}^* = \underset{\mathcal{U} \subseteq S_k, |\mathcal{U}|=N}{\operatorname{argmax}} \sum_{i \in \mathcal{U}} -H_k(i),$$

recovering the unmasking rule of KLASS.

Proof. Δ_{S_k} is itself a probability simplex, supported on $J(S_k)$. The KKT analysis in the proof of Theorem 4.1 applies verbatim with the index j ranging over $J(S_k)$ rather than the full set, since the stationarity condition is separable across coordinates and the constraint $\pi \in \Delta_{S_k}$ preserves convexity. The optimum is therefore attained at a vertex e_{j^*} , with j^* corresponding to the size- N subset $\mathcal{U}^* \subseteq S_k$ that minimizes the total entropy $\sum_{i \in \mathcal{U}} H_k(i)$. $\square$

The same reduction applies to any confidence-based sampler that imposes a data-dependent feasibility set $\mathcal{U} \in \mathcal{C}(\mathbf{x}_k)$ determined by the model’s predictions (and not by π itself). In each case, the optimal policy under the constrained objective is the entropy-greedy choice within $\mathcal{C}(\mathbf{x}_k)$.

B.4 Proof of Theorem 4.2

Proof. The Hessian of the objective is

$$\nabla^2 f(\pi) = \mathbb{E} \left[\frac{\mathbf{z}\mathbf{z}^\top}{(\mathbf{z}^\top \pi)^2} \right] \succeq \mathbb{E}[\mathbf{z}\mathbf{z}^\top] = \Sigma,$$

for $(\mathbf{z}^\top \pi)^2 \leq \|\mathbf{z}\|_\infty^2 \leq 1$. Therefore, $\nabla^2 f(\pi)$ is positive-definite with smallest eigenvalue greater than $\lambda = \lambda_{\min}(\Sigma)$. This means f is λ -strongly convex, i.e. for any π, π' ,

$$f(\pi) - f(\pi') \geq \langle \nabla f(\pi'), \pi - \pi' \rangle + \frac{\lambda}{2} \|\pi - \pi'\|_2^2.$$

Denoting $\hat{\pi} = e_{j^*}$ for j^* corresponding to $\mathcal{U}^*$. By the above inequality we have

$$\begin{aligned} f(\hat{\pi}) - f(\pi^*) &\leq \langle \nabla f(\hat{\pi}), \hat{\pi} - \pi^* \rangle - \frac{\lambda}{2} \|\hat{\pi} - \pi^*\|_2^2 \\ &= \langle P \nabla f(\hat{\pi}), \hat{\pi} - \pi^* \rangle + \langle \nabla f(\hat{\pi}) - P \nabla f(\hat{\pi}), \hat{\pi} - \pi^* \rangle - \frac{\lambda}{2} \|\hat{\pi} - \pi^*\|_2^2 \\ &= \langle P \nabla f(\hat{\pi}), \hat{\pi} - \pi^* \rangle - \frac{\lambda}{2} \|\hat{\pi} - \pi^*\|_2^2 \end{aligned} \quad (12)$$

$$\leq \|P \nabla f(\hat{\pi})\|_2 \|\hat{\pi} - \pi^*\|_2 - \frac{\lambda}{2} \|\hat{\pi} - \pi^*\|_2^2 \leq \frac{\|P \nabla f(\hat{\pi})\|_2^2}{2\lambda}, \quad (13)$$

where (12) is due to the fact that $\hat{\pi} - \pi^*$ lies in the projection hyperplane of P , and (13) is due to Cauchy-Schwartz inequality and quadratic maximization. On the other hand, we also have

$$f(\hat{\pi}) - f(\pi^*) \geq \langle \nabla f(\pi^*), \hat{\pi} - \pi^* \rangle + \frac{\lambda}{2} \|\hat{\pi} - \pi^*\|_2^2 = \frac{\lambda}{2} \|\hat{\pi} - \pi^*\|_2^2,$$

since π^* lies in the interior of Δ and the projected gradient should vanish at optimality [40]. Combining this with (13) we have

$$\|\hat{\pi} - \pi^*\|_2 \leq \frac{\|P \nabla f(\hat{\pi})\|_2}{\lambda}.$$

Since $\hat{\pi} = e_{j^*}$,

$$1 - \pi_{j^*}^* = \hat{\pi}_{j^*} - \pi_{j^*}^* \leq \|\hat{\pi} - \pi^*\|_\infty \leq \|\hat{\pi} - \pi^*\|_2,$$

which completes the proof. $\square$

C Further Details on VLP

C.1 Model Architecture

To represent a policy π_ϕ , we use a neural network that takes current sampling state $\mathbf{x}_k$ as input, and outputs a probability distribution over the size- (L/K) index sets $\mathcal{U}_k$ for the next unmasking step. In our implementation, we adopt a shared-torso architecture: the policy network reuses the trained backbone of the MDM and appends an additional lightweight module. This reduces the number of trainable parameters: we pass the MDM’s final-layer hidden states $\mathbf{h}_k$ to the policy in place of the raw tokens $\mathbf{x}_k$. A similar parameterization applies to the distribution w .

In our implementation, the additional module is a single-layer Transformer [38] block with bidirectional multi-head attention and rotary positional embedding [35]. In addition to the hidden states $\mathbf{h}_k$, via concatenation, we also input two binary embeddings $\mathbf{v}_{\text{mask}} =$

$(\mathbf{1}_{x_k^1=\text{M}}, \dots, \mathbf{1}_{x_k^L=\text{M}}) \in \{0, 1\}^L$ and $\mathbf{v}_{\text{prompt}} = (\mathbf{1}_{x_k^1 \text{ is prompt}}, \dots, \mathbf{1}_{x_k^L \text{ is prompt}}) \in \{0, 1\}^L$ to explicitly provide the mask and prompt positions of the sequence. The output is a probability vector $\mathbf{p} = (p_1, \dots, p_L)$ through an application of the softmax function. It models a multinomial distribution over index sets, where each index i is randomly selected to join $\mathcal{U}_k$ with probability p_i . We adopt the multinomial parameterization rather than independent per-position Bernoulli draws (as in, e.g., Chen et al. [8]) since it natively models distributions over fixed-size index sets; in early experiments, an independent-Bernoulli variant collapsed toward unmasking very few tokens per step, since such low-cardinality outcomes have higher joint probability under the model. We illustrate the above design in Figure 3.

More concretely, let $N = L/K$ be the number of positions to unmask at one step. At inference time, we sample N distinct positions using `torch.multinomial` with `replacement=False`. Thus, for a size- N set $\mathcal{U}$, the induced set probability is the sequential without-replacement probability

$$\pi_{\text{norep}}(\mathcal{U}|\mathbf{x}_k) = \sum_{\sigma \in \text{Sym}(\mathcal{U})} \prod_{r=1}^N \frac{p_{\sigma_r}}{1 - \sum_{s < r} p_{\sigma_s}},$$

where the sum is over all orderings of the elements in $\mathcal{U}$. During policy training, however, directly evaluating π_{norep} is computationally prohibitive and we use the surrogate `torch.multinomial` with `replacement=True`. This yields the simple likelihood for an ordered draw $(i_1, \dots, i_N)$,

$$\pi_{\text{rep}}(i_1, \dots, i_N|\mathbf{x}_k) = \prod_{r=1}^N p_{i_r},$$

which makes the policy log-probability straightforward to compute by summing $\log p_{i_r}$. This surrogate can be bounded from both sides by the following. For a distinct set $\mathcal{U}$, the with-replacement probability of drawing exactly the elements of $\mathcal{U}$ once, in any order, is $N! \prod_{i \in \mathcal{U}} p_i$. Since every normalizing denominator in the without-replacement probability is at most one,

$$N! \prod_{i \in \mathcal{U}} p_i \leq \pi_{\text{norep}}(\mathcal{U}|\mathbf{x}_k).$$

Moreover, if $p_{\mathcal{U}} := \sum_{i \in \mathcal{U}} p_i < 1$, then each nontrivial denominator is at least $1 - p_{\mathcal{U}}$, giving the upper bound

$$\pi_{\text{norep}}(\mathcal{U}|\mathbf{x}_k) \leq \frac{N! \prod_{i \in \mathcal{U}} p_i}{(1 - p_{\mathcal{U}})^{N-1}}.$$

C.2 Training and Inference

We summarize training and inference of VLP in Algorithm 1 and 2, respectively. In general, at each training iteration we draw a data sample $\mathbf{x}_0 \sim q_0$, sample a masking trajectory from w_ϕ , and compute the corresponding unmasking loss using π_ϕ . Since ϕ also appears in the sampling of the masking trajectory from w_ϕ , the stochastic gradients $\nabla_{\phi} s^{(i)}$ must be corrected via REINFORCE [39] to obtain

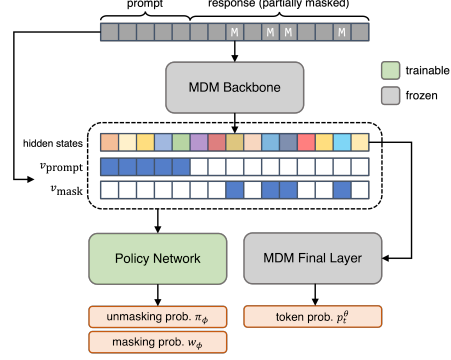


Figure 3: An illustration of the shared-torso architecture of VLP with a trained MDM.

Algorithm 1 VLP Training

Input: Initialization ϕ , trained MDM p_t^θ , dataset $\mathcal{D}$, batch size B , total sampling steps K , learning rate η , regularization strength λ

Output: Trained policy π_ϕ

```

1: repeat
2:   Sample data batch  $\{\mathbf{x}_0^{(i)}\}_{i=1}^B \sim \mathcal{D}$ 
3:   for  $i = 1, \dots, B$  do
4:     Sample  $\mathcal{U}^{(i)} \sim w_\phi(\cdot | \mathbf{x}_0^{(i)})$ 
5:     Construct  $\mathbf{x}_1^{(i)}, \dots, \mathbf{x}_K^{(i)}$  according to  $\mathcal{U}^{(i)}$ 
6:      $w^{(i)} \leftarrow w_\phi(\mathcal{U}^{(i)} | \mathbf{x}_0^{(i)})$ 
7:      $p^{(i)} \leftarrow \prod_{k=1}^K p_{t_k}^\theta(\mathbf{x}_{k-1}^{(i)} | \mathbf{x}_k^{(i)}) \pi_\phi(\mathcal{U}_k^{(i)} | \mathbf{x}_k^{(i)})$ 
8:      $r^{(i)} \leftarrow \sum_{k=1}^K \text{KL}(\pi_\phi(\cdot | \mathbf{x}_k^{(i)}) \| \pi_{\text{ref}}(\cdot | \mathbf{x}_k^{(i)})) + \text{KL}(w_\phi(\cdot | \mathbf{x}_0^{(i)}) \| w_{\text{ref}}(\cdot | \mathbf{x}_0^{(i)}))$ 
9:      $s^{(i)} \leftarrow \log w^{(i)} - \log p^{(i)} + \lambda r^{(i)}$ 
10:   end for
11:    $s \leftarrow \frac{1}{B} \sum_{i=1}^B s^{(i)}$ 
12:    $g_{\text{REINFORCE}} \leftarrow \frac{1}{B} \sum_{i=1}^B [\nabla_\phi s^{(i)} + (s^{(i)} - s) \nabla_\phi \log w^{(i)}]$ 
13:    $\phi \leftarrow \phi - \eta g_{\text{REINFORCE}}$ 
14: until converged

```

Algorithm 2 VLP Inference

Input: Policy π_ϕ , trained MDM p_t^θ , total sampling steps K , sequence length L

Output: Generated sample $\mathbf{x}_0$

```

1:  $\mathbf{x}_K \leftarrow (\mathbf{M}, \mathbf{M}, \dots, \mathbf{M})$ 
2: for  $k = K, \dots, 1$  do
3:   Sample  $\mathcal{U}_k \sim \pi_\phi(\cdot | \mathbf{x}_k)$ 
4:   Sample candidate tokens  $\hat{\mathbf{x}}_{k-1} \sim p_{t_k}^\theta(\cdot | \mathbf{x}_k)$ 
5:   Construct  $\mathbf{x}_{k-1}$  elementwise by

```

$$x_{k-1}^i \leftarrow \hat{x}_{k-1}^i \mathbf{1}_{i \in \mathcal{U}_k} + x_k^i \mathbf{1}_{i \notin \mathcal{U}_k}$$

```

6: end for
7: return  $\mathbf{x}_0$ 

```

an unbiased estimate $g_{\text{REINFORCE}} = \nabla_\phi s^{(i)} + (s^{(i)} - s) \nabla_\phi \log w^{(i)}$. While REINFORCE is known to have high gradient variance [31], we retain this estimator to maintain methodological compatibility with Wang et al. [39], whose framework our work extends to multi-token unmasking. Lower-variance alternatives are left to future work. At the inference stage, we directly use the trained policy to sample an unmasking index set and apply the trained MDM to predict tokens.

D Further Details on Experiments

D.1 Optimality Analysis

D.1.1 Synthetic Experiments

We use a small synthetic setting to visualize the likelihood-optimization landscape for unmasking policies. The sequence length is set to $L = 3$, the vocabulary size to $V = 10$, and the number of unmasked positions selected at each step to $N = 2$. We generate a product data distribution by sampling token logits independently and applying position-dependent temperatures (0.1, 0.5, 2.0), followed by a softmax normalization. This creates positions with different uncertainty levels.

For each policy, we evaluate the objective exactly by enumerating all clean sequences in $[V]^L$ and all possible N -hot unmasking masks. Since positions are sampled without replacement, the probability of each unordered mask is computed exactly by dynamic programming over all possible sampling

orders. We compare uniform sampling, greedy confidence sampling, and randomized confidence sampling. The confidence scores are based on negative entropy, top-token probability, and top-2 probability margin. For randomized confidence policies, we apply a softmax over the scores with various temperatures $T \in \{0.1, 0.2, \dots, 2.0, 3, 5, 8, 10\}$.

To find an approximate optimum, we enumerate a regular grid over the policy simplex and evaluate the exact objective at each grid point. Each policy is mapped to its induced distribution over the 3 possible 2-position masks, yielding a visualization on the mask-probability simplex. We then plot the objective landscape, the optimizer, the uniform and greedy confidence methods, and a continuum of random confidence methods as the temperature T varies. In Figure 4, we provide more landscape visualization plots under different random seeds for the synthetic data generation.

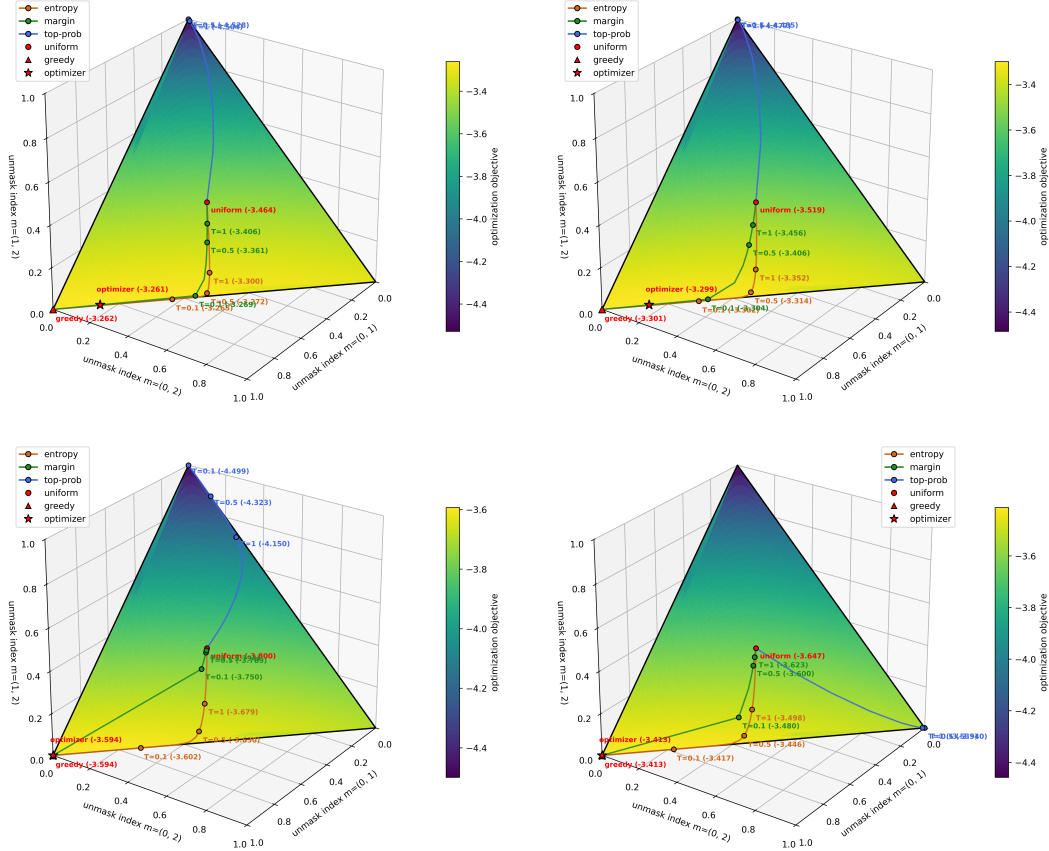


Figure 4: Policy simplex and different unmasking strategies under various choices of data-generating seed in the synthetic data.

D.1.2 Text Generation Experiments

We evaluate the objective value induced by different unmasking strategies during diffusion-based text generation. Samples are generated using the pretrained RADD model [26]. The generation length is fixed to 1024 tokens with batch size 4, and each experiment is repeated over 100 independent sampling runs. We vary the number of sampling steps $N \in \{128, 256, 512, 1024\}$. At each sampling step i , the sampler unmask 1024/ N token positions. We compare three strategies: uniform unmasking, greedy confidence based on top-2 probability margin, and its randomized version.

For each run and sampling step, we estimate the objective value (7) using Monte Carlo approximation. Given the model conditional token distribution at the current partially masked sequence, we sample complete token proposals and random unmasking subsets. The objective is estimated from the log probability of selecting the subset together with the conditional token likelihood, with a combinatorial correction for the number of possible subsets. In the implementation, we use 10 Monte Carlo token

samples and 10 sampled unmasking subsets per step. For the greedy case, the objective is computed using the top selected negative-entropy terms.

For each N , we aggregate the objective values across up to 100 runs for each strategy in Figure 5. Faint curves display the raw per-run objective trajectories, while solid curves show the mean objective value at each sampling step.

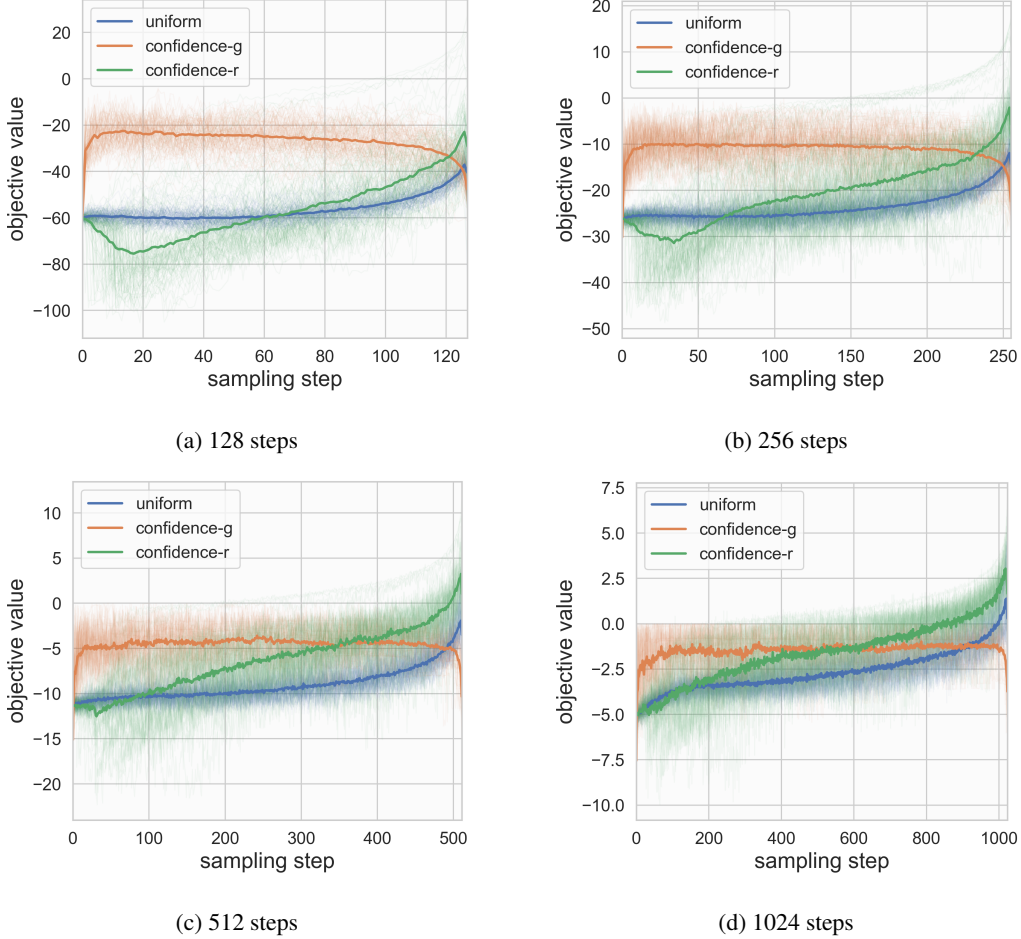


Figure 5: Local objective values (7) attained by different unmasking methods in text generation with RADD [27].

D.2 Evaluation of VLP

D.2.1 Experiment Setup

Text Generation Details. For the RADD text generation experiments, we use the pretrained checkpoint corresponding to the small RADD configuration with 12 Transformer blocks, hidden size 768, 12 attention heads, sequence length 1024, and approximately 162M parameters. The backbone is kept frozen throughout policy training. The policy network follows Section C.1, with approximately 5.3M trainable parameters, about 3% of the RADD backbone.

We train the policy on OpenWebText. Text is tokenized with the GPT-2 tokenizer, appended with EOS tokens, concatenated, and split into fixed-length chunks of 1024 tokens. Since this is unconditional generation, there is no prompt portion and all positions are eligible for masking or unmasking. Unless otherwise specified, policy training uses batch size 4, one gradient accumulation step, and $K = 32$ discrete masking steps for the RADD runs used in our reported sampling experiments. We optimize with AdamW using learning rate 3×10^{-4} , weight decay 0, $(\beta_1, \beta_2) = (0.9, 0.999)$,

$\epsilon = 10^{-8}$, gradient clipping at norm 1.0, and EMA decay 0.9999 for the policy parameters. The KL regularization parameter λ is set to 10^{-2} . Training is run on a single A100 80GB GPU for 2×10^3 steps.

For evaluation, we generate unconditional sequences of length 1024 using the RADD diffusion sampler. We evaluate sampling budgets from 16 to 256 steps. For each setting, we generate 100 independent samples and report the mean and standard deviation of GPT-2-large perplexity. We compare uniform unmasking, greedy margin-based confidence unmasking, randomized margin-based confidence unmasking, and the learned policy.

Code and Math Generation Details. For both 7B models, we evaluate HumanEval [7], MBPP [2], GSM8K [10], and MATH500 [19]. HumanEval and MBPP are evaluated using Pass@1 ($\uparrow$), while GSM8K and MATH500 are evaluated using final-answer accuracy ($\uparrow$). All evaluations are run through the LM Evaluation Harness with our DiffuCoder wrapper.

The policy is trained on instruction-output pairs from OpenCoder-SFT [12] and MetaMathQA [43]. Each example is converted to the chat format shown in Table 8. We tokenize each formatted sequence with the corresponding model tokenizer, truncate or pad to length 1024, and construct a prompt mask marking all tokens before the assistant response. During training and inference, prompt tokens are kept fixed; only response tokens are masked, unmasked, or generated.

The backbone model is loaded in `bfloat16` and kept frozen throughout policy training. The trained policy is a lightweight transformer policy network as described in Section C.1, with approximately 115.7M parameters or about 1.5% of the 7.6B-parameter backbone.

Unless otherwise specified, we train the policy with batch size 4, sequence length 1024, and $K = 16$ discrete masking steps. We optimize the policy with AdamW using learning rate 10^{-3} , weight decay 0, $(\beta_1, \beta_2) = (0.9, 0.999)$, $\epsilon = 10^{-8}$, and gradient clipping at norm 1.0. The KL regularization parameter λ is set to 10^{-2} . Training is run on a single A100 80GB GPU for 10^3 steps.

For evaluation, we use zero-shot setting (no in-context demonstrations) and prompts for HumanEval, MBPP, and GSM8K are formatted with the chat template (Table 9, 10, 11). All prompts are padded with mask tokens up to the requested generation length. We use context length 1024 for code tasks (HumanEval, MBPP) and 512 for math tasks (GSM8K, MATH500). Generation uses the diffusion sampler for the specified number of steps. For confidence and policy variants, an algorithm temperature controls whether positions are selected greedily or sampled from a softmax over scores. Unless otherwise specified, the temperature for performing categorical sampling among the vocabulary is set to 0.

D.2.2 Additional Results

RADD Table 3 evaluates the mean and standard deviation of GPT-2 perplexity as well as sample entropy in unconditional generations from RADD.

Table 3: Mean GPT-2 perplexity ($\downarrow$), perplexity standard deviation, and sample entropy of unconditional generations from RADD under different unmasking strategies and numbers of sampling steps. All generations have context length 1024. OpenWebText dataset has sample entropy 5.44.

steps	Perplexity mean					Perplexity std. dev.					Entropy				
	16	32	64	128	256	16	32	64	128	256	16	32	64	128	256
uniform	330.3	192.1	145.5	134.5	116.8	100.6	54.5	47.4	46.7	60.0	5.77	5.71	5.68	5.69	5.64
confidence-g	1123.0	1036.5	982.4	860.7	550.8	290.8	333.5	330.6	335.4	257.0	5.72	5.64	5.59	5.60	5.52
confidence-r	313.0	181.6	140.7	116.7	103.4	78.2	53.3	50.5	41.1	35.3	5.74	5.67	5.63	5.61	5.59
VLP	300.8	174.8	138.0	128.9	106.7	77.4	43.7	38.1	45.4	47.8	5.78	5.72	5.67	5.67	5.63

Evaluation on DiffuCoder-7B. Table 4 reports the corresponding full DiffuCoder-7B results on HumanEval, MBPP, GSM8K, and MATH500. VLP consistently improves over uniform unmasking and is able to achieve matching or better performance compared to confidence methods. Note that the backbone DiffuCoder-7B model is finetuned specifically on coding performance [13], leading to a degrade of math abilities regardless of decoding methods.

Table 4: Results ($\uparrow$) of DiffuCoder-7B under different unmasking strategies and numbers of sampling steps across four benchmarks.

steps	HumanEval				MBPP				GSM8K				MATH500			
	16	32	64	128	16	32	64	128	16	32	64	128	16	32	64	128
uniform	15.9	23.8	22.6	28.7	23.8	31.6	35.8	39.4	7.1	7.3	8.8	9.1	3.8	4.6	5.4	6.4
confidence-g	4.3	11.6	26.2	30.2	1.8	5.0	18.4	31.2	2.4	3.8	6.4	6.4	1.4	1.4	2.4	3.0
confidence-r	20.7	25.0	29.9	32.3	33.2	39.4	39.8	41.8	7.3	10.2	10.5	10.4	4.2	5.8	5.0	6.0
VLP	23.2	27.4	30.5	33.5	29.8	37.6	40.4	42.4	7.4	9.7	10.2	10.4	6.4	8.8	8.2	7.0

Effect of KL regularization in VLP training. Table 5 studies the effect of the KL regularization strength λ in RADD text generation with diffusion steps K for training fixed at 16. We evaluate unconditional generation quality using perplexity and entropy across different sampling steps. The best value of λ depends on the inference budget: $\lambda = 10^{-2}$ gives the lowest perplexity at 16 steps, while $\lambda = 10^{-1}$ gives the lowest perplexity at 32, 64, and 128 steps. This suggests that KL regularization consistently brings improvement to generation quality.

Table 5: Effect of KL regularization strength λ in VLP training for RADD. We report mean GPT-2 perplexity ($\downarrow$), perplexity standard deviation, and entropy of unconditional generation at different sampling steps. OpenWebText dataset has sample entropy 5.44.

λ	Perplexity mean				Perplexity std. dev.				Entropy			
	16	32	64	128	16	32	64	128	16	32	64	128
0	313.9	186.0	142.5	127.4	85.2	47.5	42.8	42.6	5.79	5.73	5.68	5.68
10^{-3}	320.0	182.8	140.5	129.8	85.7	41.0	35.6	47.5	5.78	5.73	5.68	5.66
10^{-2}	301.7	186.8	145.3	127.3	81.5	48.3	47.9	47.9	5.77	5.73	5.68	5.67
10^{-1}	301.8	179.1	134.4	118.6	89.4	42.6	41.9	35.0	5.77	5.72	5.67	5.64

Effect of sampling steps K in VLP training. Table 6 ablates the choice of the training diffusion step K for RADD, with λ fixed at 10^{-2} . Although each policy is trained with a fixed K , we evaluate it across several sampling budgets. Training with $K = 32$ gives the strongest overall perplexity, suggesting that a mid-range training horizon transfers well across different numbers of inference steps.

Table 6: Effect of choice of K in VLP training for RADD. We report mean GPT-2 perplexity ($\downarrow$), perplexity standard deviation, and entropy of unconditional generation at different sampling steps. OpenWebText dataset has sample entropy 5.44.

K	Perplexity mean				Perplexity std. dev.				Entropy			
	16	32	64	128	16	32	64	128	16	32	64	128
16	301.7	186.8	145.3	127.3	81.5	48.3	47.9	47.9	5.77	5.73	5.68	5.67
32	300.8	174.8	138.0	128.9	77.4	43.7	38.1	45.4	5.78	5.72	5.67	5.67
64	316.2	178.1	141.8	125.5	83.2	44.0	45.2	42.9	5.78	5.72	5.69	5.66

Effect of sampling temperature. The generation process involves two types of temperature: (1) vocabulary temperature that controls the categorical sampling among the vocabulary, and (2) policy temperature that controls the greediness of choosing unmasking indices from the predicted multinomial distribution. Table 7 studies the effect of varying both types of temperatures on Pass@1 score of HumanEval with DiffuCoder-7B model. The results show that low vocabulary temperature is important for reliable code generation, while a moderate policy temperature can improve Pass@1 by avoiding overly deterministic position selection.

Table 7: Temperature analysis on HumanEval. We report Pass@1 ($\uparrow$) at different sampling steps.

Temperature type	Value	Steps	
		16	32
Vocabulary	0.0	23.2	25.0
	0.1	18.9	32.3
	0.2	19.5	28.0
	0.5	15.2	22.6
	0.8	11.0	22.6
	1.0	7.9	11.6
Policy	0.1	13.4	24.4
	0.2	18.9	25.0
	0.3	20.7	26.8
	0.4	25.0	27.4
	0.5	23.2	27.4
	0.6	22.6	23.2
	0.7	21.3	22.0
	0.8	20.7	25.0
	0.9	18.3	23.2
	1.0	17.1	24.4

D.2.3 Datasets, Models, and Licenses

Table 12 lists the public datasets and pretrained model checkpoints used in our experiments, together with their public URLs and license information. License names are taken from the corresponding Hugging Face model or dataset cards when available.

Table 8: Chat template for VLP OpenCoder-SFT training.

```
<|im_start|>system
You are a helpful
assistant.<|im_end|>
<|im_start|>user
{query}
<|im_end|>
<|im_start|>assistant
{output}
<|im_end|>
```

Table 9: Chat template for VLP HumanEval evaluation.

```
<|im_start|>system
You are a helpful
assistant.<|im_end|>
<|im_start|>user
Please complete the following
problem:
```
{prompt}
```
<|im_end|>
<|im_start|>assistant
Here is the code to solve this
problem:
```python
```

Table 10: Chat template for VLP MBPP evaluation.

```
<|im_start|>system
You are a helpful
assistant.<|im_end|>
<|im_start|>user
```
{prompt}
```
<|im_end|>
<|im_start|>assistant
Here is the code to solve this
problem:
```python
```

Table 11: Chat template for VLP GSM8K evaluation.

```
<|im_start|>system
You are a helpful
assistant.<|im_end|>
<|im_start|>user
{question}
<|im_end|>
<|im_start|>assistant
Let's think step by step.
```

Table 12: Public datasets and pretrained models used in the experiments.

Asset	Type	URL	License
OpenWebText	Dataset	https://huggingface.co/datasets/Skylion007/openwebtext	CC0-1.0
OpenCoder-SFT stage 2	Dataset	https://huggingface.co/datasets/OpenCoder-LLM/opc-sft-stage2	MIT
MetaMathQA	Dataset	https://huggingface.co/datasets/meta-math/MetaMathQA	MIT
RADD	Model	https://huggingface.co/JingyangOu/radd-lambda-dce	Apache-2.0 for official code release; no license specified on the Hugging Face model card
Dream-7B-Base	Model	https://huggingface.co/Dream-org/Dream-v0-Base-7B	Apache-2.0
DiffuCoder-7B-Instruct	Model	https://huggingface.co/apple/DiffuCoder-7B-Instruct	Apple Machine Learning Research License (apple-amlr)